



FACULTY OF SCIENCE

ACADEMY OF COMPUTER SCIENCE AND SOFTWARE ENGINEERING

MODULE	COMPUTER SCIENCE 3B CSC03B3
CAMPUS	AUCKLAND PARK CAMPUS (APK)
SUMMATIVE ASSESSMENT OPPORTUNITY 1	SAO1

DATE: 2025-08-29	SESSION: 14h00 - 16h00
ASSESSOR(S):	MR A MAGANLAL
	MR C KABUYA
MODERATOR:	INTERNAL: DR S SITHUNGU
DURATION: 120 MINUTES	MARKS: 100

Please read the following instructions carefully:

1. You are bound by all university regulations including, but not limited to assessment, plagiarism, and ethical conduct.
 2. You may not leave the venue in the first 30 minutes of the paper.
 3. You may not leave the venue in the last 15 minutes of the paper.
 4. Answer **all** the questions in *sequential* order.
 5. Clearly indicate which question is being answered.
 6. Answer only in the answer sheets provided.
 7. The use of calculators is *not* permitted.
 8. Write ***cleanly and legibly***.
 9. This paper contains **7** questions.
 10. This paper consists of **3** pages, excluding the cover page.
-

QUESTION 1: Operating Systems - General

- (a) **Name** and **describe** the four (4) *components* of a **computer system**. [08]
- (b) **Name** and **discuss** the *role* each of the four (4) **storage-devices** play when a program is **executed**. The storage-devices are part of the **storage-device hierarchy**. [08]
- (c) **Provide** a **definition** of each of the following *terms*:
- Volatile memory [02]
 - Program Counter [02]

Total: 20

QUESTION 2: Operating System Structures

- (a) **Describe** two (2) *different* methods used to pass parameters in **system calls**. [04]
- (b) **Analyse** each of the following statements. Clearly state whether the statement is **TRUE** or **FALSE** and **motivate** your answer for either case.
- Operating systems** are *written* purely in **assembly language**. [02]
 - An operating system structure that uses loadable kernel modules is a form of a layered design. [02]
 - System calls can pass parameters using a register. [02]
- (c) **'INVOKE ExitProcess, 0'** is a system call function that should be the last instruction in **CSC3B** assembly programs. **Describe** how the **ExitProcess** system call executes in the operating system. [05]

Total: 15

QUESTION 3: Processes

- (a) **Analyse** the table below that *represents process control blocks (PCBs)* and **answer** questions that follow. Each entry in the **PCB** takes up **4** bytes.

State	READY	RUNNING	WAITING	READY
Name	browser.exe	7x.exe	backup.exe	selfserv.exe
PID	24859	21156	18479	16234
PC	2000DE08	10004F56	01004D17	00100079
EAX	000D2CD5	10008B5A	01004D17	00104BD9
EBX	2000DE08	00092A5B	0008D272	00100079
ECX	0008EE73	000D369A	0002FD7F	000469FF
EDX	000DE9B6	0009C775	00032FD4	000919B9
ADDR Start	20000000	10000000	01000000	00100000
ADDR End	20010000	10010000	01010000	00110000

- Provide** the *process ID* that is currently being used by the processor. [01]
- Provide** the *range of memory addresses* used by the **backup.exe** process. [01]
- Provide** the *process ID* that is **blocked** on *IO*. [01]
- Assuming that the **scheduler** selects processes according to ascending PID, **name** the *process* that will run next. [01]
- Calculate** the number of bytes that are *saved* and *loaded* during a context switch. [01]

~~ Assessment continues on the next page. ~~

- (b) **Compare** the *stack* to the *heap*. [04]
- (c) **Name** and **motivate** the *process state* for each of the following scenarios:
- A word processor is loading a document. [02]
 - A word processor has just finished loading a document. [02]
 - A word processor is scanning a document for spelling mistakes. [02]

Total: 15

QUESTION 4: Threads

- (a) **Discuss** the *differences* between *data parallelism* and *task parallelism*. [04]
- (b) **Discuss** the *benefits* of *economy* and *scalability* in thread usage. [04]
- (c) **Analyse** each of the following statements. Clearly state whether the statement is **TRUE** or **FALSE**, and **motivate** your answer for either case.
- Implicit threading makes it easier for developers to use threads. [02]
 - A thread pool creates a thread in the pool when a thread task is invoked. [02]
- (d) For each of the following items, **specify** whether the item is associated with a per-thread level or a per-process level, e.g. x) Thread.
- Registers [01]
 - Global variables [01]
 - Accounting information [01]

Total: 15

QUESTION 5: Process Synchronisation

- (a) **Describe** how *no busy waiting semaphores* work. [04]
- (b) **Name** two (2) *disadvantages* of *semaphores*. [02]
- (c) **Draw** a *resource allocation graph* for the following states **and** specify whether the system is in a deadlock: [04]
- | | |
|--|--|
| <ul style="list-style-type: none"> • Process A holds X and requests Y • Process B holds Y and requests Z | <ul style="list-style-type: none"> • Process C requests X and W • Process D holds Z and requests W |
|--|--|

Total: 10

QUESTION 6: Number Systems

Show all calculation steps!

- (a) **Hexadecimal** and **octal** can be seen as shorthand for binary. **Solve** the following equation, which contains terms in multiple number systems and provides the answer in **hexadecimal** (e.g. $q = 17_{16}$): [05]
- $$x_{16} = 00001000_2 + 45_8 + 78_{16}$$
- (b) **Convert** the number below into *two's complement* binary representation. Use exactly 8-bits to represent the number. Show all the steps in the calculation. [05]

$$-109_{10}$$

Total: 10

~~ Assessment continues on the next page. ~~

QUESTION 7: 80x86 Assembly

Provide code for 80x86 MASM assembly function that is equivalent to the following **C++** function:

```
1 | void grip(int str, int* arr, int len)
2 | {
3 |     for (int x = 0; x < len; x++)
4 |     {
5 |         str *= arr[x];
6 |     }
7 | }
```

Only provide code for the function. No global variables are to be used.

Total: 15

❖ THE END ❖